

WEB APPLICATION BUILD SUMMARY

Wrist Mode Website

A simple guide to the tools, files, features, and systems used to build the watch and customized jewelry website.

Quick summary: The application is a local dynamic storefront and admin dashboard for Wrist Mode. It is built with Node.js, Express, SQL.js, plain HTML/CSS/JavaScript, Three.js for the 3D custom studio, and local image assets for products and mockups.

1. Purpose Of The Website

The website was built to present Wrist Mode as a watch and customized jewelry brand. It lets customers browse watches, women watches, wooden watches, ready jewelry, and custom jewelry options. It also supports cart orders, custom design requests, order tracking, and an admin area for managing products and customer requests.

Business focus: Watches, women watch sets, wooden watches, bar necklaces, pendants, bracelets, bangles, keyholders, dog tags, and personalized photo or text jewelry.

Current local address: <http://localhost:3000> when the server is running.

2. Main Technologies Used

Technology	Used For	Where It Appears
Node.js	Runs the website server and scripts.	package.json, server.js
Express	Creates the web server, static file serving, and API routes.	server.js
SQL.js / SQLite file	Stores products, orders, custom requests, stock alerts, and analytics data locally.	data/wrist-mode.sqlite
Multer	Handles uploaded images for admin product photos and customer custom-reference images.	server.js, uploads folder
HTML	Provides the main browser page structure.	public/index.html
CSS	Controls the dark luxury look, responsive layout, cards, admin panels, product grids, and customizer styling.	public/styles.css
JavaScript	Runs the storefront views, cart, filters, forms, admin screens, and API calls.	public/app.js

Technology	Used For	Where It Appears
Three.js	Powers the interactive 3D custom jewelry studio with orbit/zoom controls and live material/text updates.	public/studio3d.js, public/vendor/three
Playwright	Used for smoke tests, mobile checks, API checks, and quality sweeps.	scripts/*.js

3. Main Files And Folders

File Or Folder	Purpose
server.js	Main backend file. It starts Express, loads the SQL.js database, exposes APIs, handles uploads, and serves the site.
package.json	Lists project scripts and dependencies such as Express, Multer, SQL.js, Three.js, and Playwright.
public/index.html	The browser entry page for the single-page website.
public/app.js	Main frontend application logic for pages, product rendering, cart, custom requests, admin views, and tracking.
public/studio3d.js	Interactive Three.js custom jewelry studio for bar, dog tag, bracelet, name necklace, envelope, book pendant, keyholder, cuff, and related pieces.
public/styles.css	Visual design, layout, mobile responsiveness, dark green/gold brand feel, galleries, forms, and admin styling.
public/assets	Local images for logos, hero images, watch categories, jewelry, custom studio mockups, and catalog fillers.
uploads	Customer/admin uploaded files, especially custom reference images and product uploads.
data/wrist-mode.sqlite	Local database file that persists store data.
scripts	Import, preparation, smoke test, API test, mobile test, QA sweep, and catalog update scripts.

4. Frontend Features

Area	What Customers See Or Do
Home	Brand introduction, product entry cards, Wrist Mode look section, and navigation to shopping/customizing areas.
Watches	Men watch catalog with filters for search, brand, availability, and price range.
Women Watches	Women watch sections arranged for plain watches, bracelet sets, and larger gift sets.
Wooden Watches	Separate section for wooden watch products and product details.

Area	What Customers See Or Do
Jewelry	Ready jewelry such as bar necklaces, pendants, bracelets, bangles, and non-customized pieces.
Customize	Interactive custom studio where customers choose a piece, finish, side, font, chain style, text, and photo upload when needed.
Product Details	Detailed view for an item with images, description, stock status, and action buttons.
Cart	Stores selected items before checkout request is submitted.
Track Order	Lets customers search their order/custom request status using the generated reference code.

5. Backend And API

The backend is built in server.js with Express. It serves the frontend files, stores uploaded images, manages admin sessions, reads and writes the SQL.js database, and returns JSON data to the frontend.

API Group	Examples	Purpose
System	GET /api/health	Checks whether the server is running.
Products	GET /api/products, GET /api/brands, GET /api/products/:id	Loads catalog data for the storefront and filters.
Admin Login	POST /api/admin/login, POST /api/admin/logout, GET /api/admin/session	Protects the admin dashboard.
Admin Products	POST /api/admin/products, PUT /api/admin/products/:id, DELETE /api/admin/products/:id	Adds, updates, and removes products from the catalog.
Orders	POST /api/orders, GET /api/track	Creates normal cart orders and lets customers track them.
Custom Requests	POST /api/custom-orders, GET /api/admin/custom-requests, PATCH /api/admin/custom-requests/:id	Handles personalized jewelry requests and admin updates.
Admin Reports	GET /api/admin/analytics, GET /api/admin/orders, GET /api/admin/notifications	Shows order activity, custom requests, and admin notifications.

6. Database And Stored Information

The site uses SQL.js, which stores the database as a local SQLite-style file. The default file is data/wrist-mode.sqlite. The server loads it when it starts and writes changes back to disk when products, orders, stock alerts, or custom requests change.

Stored Data	Examples
Products	Name, brand, category, price, stock status, description, image list, tags, and product options.
Orders	Customer details, cart items, totals, order status, quote/message fields, and tracking references.
Custom Requests	Jewelry type, selected design piece, finish, side, font, chain style, engraving text, uploaded photo/reference image, notes, status, quote, and admin message.
Stock Alerts	Customer requests to be notified about a product or style.
Admin Notifications	New orders, custom requests, and activity that the admin dashboard needs to show.

7. Images, Uploads, And Product Media

Product and mockup images are stored locally so the website can display them without depending on a remote image service. The current asset folders include watch photos, women watch photos, wooden watches, custom jewelry, ready jewelry, catalog fillers, brand logos, and custom studio mockups.

Media Location	Used For
public/assets/wrist-mode-logo.png and wrist-mode-mark.png	Brand logo and navigation/identity marks.
public/assets/watch-hero.jpg	Main visual image for the home/hero area.
public/assets/products/watches	Men and mixed watch product photos.
public/assets/products/women-watches	Women watches, bracelet sets, and gift set images.
public/assets/products/wooden-watches	Wooden watch product images.
public/assets/products/custom-jewelry	Customized jewelry examples and product photos.
public/assets/custom-studio	Clean jewelry mockups used by the custom studio and preview cards.
uploads	Images uploaded through admin or by customers when making custom requests.

8. 3D Custom Jewelry Studio

The custom studio is built with Three.js in `public/studio3d.js`. It creates simple 3D stand-in jewelry models that can later be replaced with real glTF or scanned jewelry models. The customer can rotate, zoom, change finish, change piece type, add engraving text, and upload a photo for picture dog tags.

Studio Feature	How It Works
3D viewport	Three.js renders the jewelry model on a canvas with camera, lights, environment, and OrbitControls.
Material finishes	Gold, silver, rose gold, black, and two-tone style materials are applied to the model.
Live engraving	CanvasTexture is used to place typed text or photo previews onto the 3D jewelry surface.
Piece switching	The frontend sends the selected piece and options to the 3D studio, which rebuilds the matching 3D stand-in model.
Customer options	Piece, variant, finish, side, font, chain style, text position/size, photo upload, and notes are collected in the customize form.

9. Admin Dashboard

The admin dashboard is part of the same website. It is protected by an admin password, then shows product management, normal orders, custom requests, analytics, and notifications. Custom orders were improved so the admin can see the requested jewelry details and customer image more clearly.

Default local admin password: `wristmode2026`. For real business use, set `ADMIN_PASSWORD` before starting the server.

Admin can manage: Products, order statuses, customer messages, custom request quotes, custom request statuses, and uploaded reference images.

10. Testing And Quality Checks

Command	Purpose
<code>npm run check</code>	Checks JavaScript syntax for <code>server.js</code> and <code>public/app.js</code> .
<code>npm run smoke</code>	Runs a basic site smoke test.
<code>npm run smoke:mobile</code>	Runs a mobile-focused smoke test.
<code>npm run smoke:api</code>	Checks important backend API behavior.
<code>npm run qa</code>	Runs the broader Playwright quality sweep.

11. How To Run The Website Locally

Step	Command Or Action
Install dependencies	npm install
Start the server	npm start
Open the website	http://localhost:3000
Use a different port	Set PORT before starting, for example PORT=3001.
Change admin password	Set ADMIN_PASSWORD before starting the server.
Change database location	Set DB_FILE if the database should be stored somewhere else.
Change upload location	Set UPLOAD_DIR if uploaded files should be stored somewhere else.

12. Maintenance Notes

The site is designed so products and images can continue growing without rebuilding the whole application. Product photos should be added to the correct asset folder, imported through the provided scripts when needed, then checked in the storefront and admin dashboard.

Area	Note
Security	Before real public use, change ADMIN_PASSWORD and avoid keeping the default password.
Payments	Payment and customer-notification buttons are provider-ready placeholders. Connect MTN Mobile Money, Airtel Money, card, email, SMS, or WhatsApp Business before accepting live online payments.
Images	Use owned, licensed, or approved product images for public launch. Keep images optimized so the site stays fast.
3D models	Current jewelry models are working stand-ins. Real scanned or modeled glTF assets can be plugged into the Three.js studio later.
Backups	Back up data/wrist-mode.sqlite and uploads regularly because they hold business records and customer-request files.
Performance	Keep product images compressed, avoid oversized gallery images, and run the smoke/QA scripts after major changes.